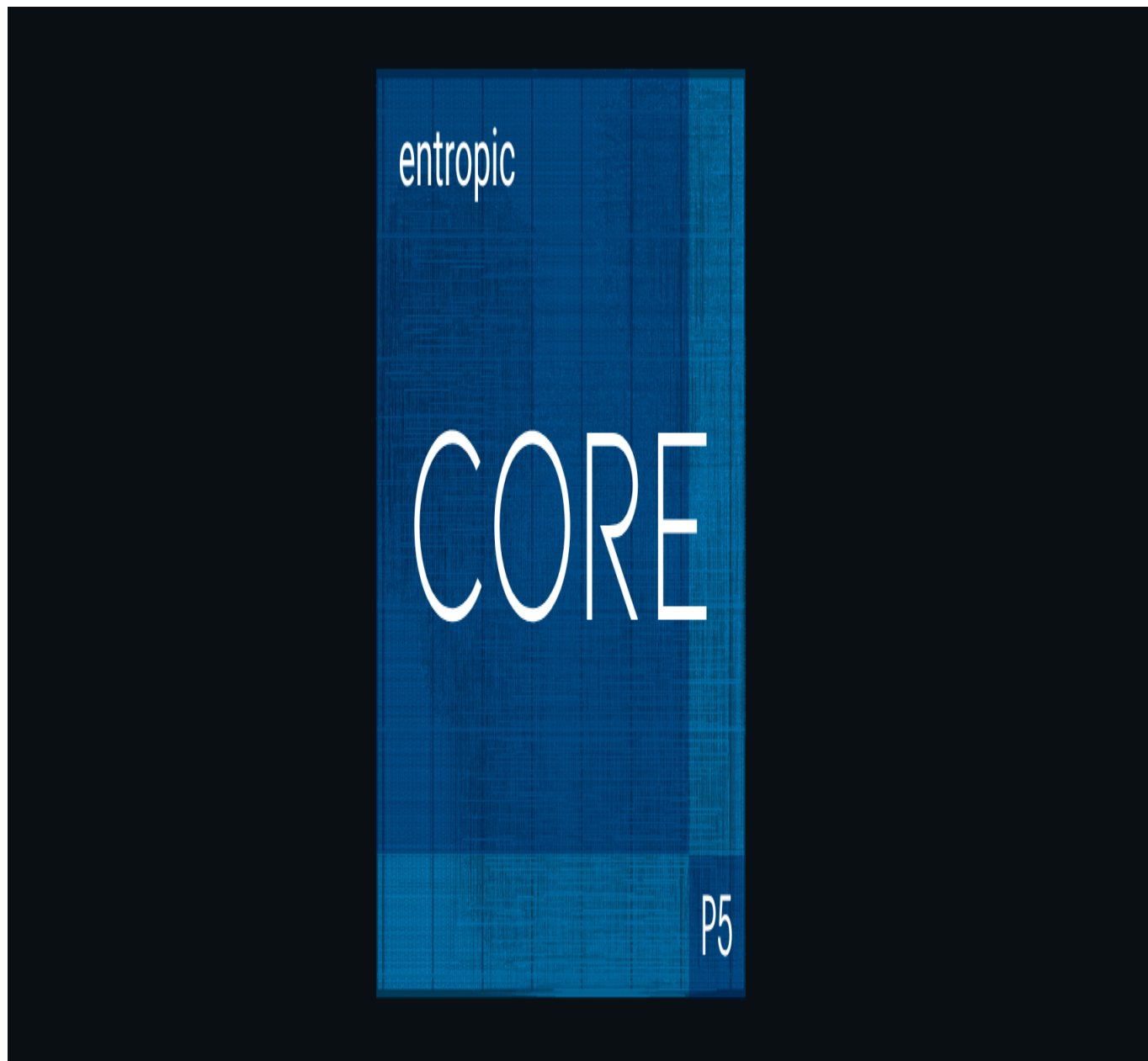




Entropic R32-P5 (RV32I 5-Stage Pipelined CPU)

(image above is the real GDS render of the CPU on a SKY130 ASIC, Intel pls don't sue me for logo I'm just a fanboy 🤪)



Entropic R32-P5 is a 5-stage pipelined **RISC-V** (RV32I) CPU, the successor to my single-cycle [Entropic R32-SC](#), that was:

- **designed** and **built completely from scratch** in **Verilog**, converting the original single-cycle datapath into a 5-stage IF/ID/EX/MEM/WB pipeline

- extended with **full data forwarding**, **hazard detection/stalling**, **control-hazard flushing with branch resolution in ID**, and a **dynamic 2-bit saturating-counter branch predictor with a 64-entry BTB**
- **fully verified** through **self-written testbenches** in **SystemVerilog** and **RISC-V Assembly**, co-driven by **cocotb**, plus **compiled C programs onto the CPU** for performance testing
- **synthesized** to a **physical ASIC layout** through **OpenLane2** on the **SkyWater 130nm** open-source PDK, achieving **50 MHz**, a ~40% clock speed improvement over the single-cycle core
- **throughput** comes out to be about **48.03 million** instructions per second for average real-world performance, and **39.37 million** instructions per second under worst-case scenarios (calculated with clock speed from [Results](#) and data from [Performance Benchmarks](#))

R32-P5 = RV32I, Pipelined, 5 stages.

Table of Contents

- [Overview](#)
 - [Architecture](#)
 - [GDS Render](#)
 - [Design](#)
 - [Pipeline Hazards & Forwarding](#)
 - [Branch Prediction](#)
 - [Real Halt Implementation](#)
 - [Verification](#)
 - [Performance Benchmarks](#)
 - [Major Debugging Findings](#)
 - [ASIC Implementation](#)
 - [Repository Structure](#)
 - [Instruction Set Coverage](#)
 - [Future Plans](#)
 - [License](#)
 - [Resources Used](#)
-

Overview

Entropic R32-P5 takes the fully-verified single-cycle Entropic R32-SC and reworks it into a standard 5-stage pipeline (IF → ID → EX → MEM → WB), then builds out everything a real pipeline needs to stay both correct and fast: full operand forwarding, load-use hazard detection with stalling, control-hazard flushing, branch resolution moved into ID for a 1-cycle misprediction penalty, and a from-scratch dynamic branch predictor.

Every submodule from the single-cycle core ([alu.v](#), [alu_control.v](#), [branch_comp.v](#), [control_unit.v](#), [imm_gen.v](#), [load_filter.v](#), [store_mask.v](#)) carries over unmodified. The pipeline is built entirely by adding pipeline registers, forwarding/hazard logic, and a predictor around the existing, already-verified datapath pieces.

The chip features:

- Full RV32I instruction coverage, same as the single-cycle

- Each pipeline capability built and verified as its own phase, each with a dedicated self-checking assembly program that checks **all pipeline hazards / forwarding behavior / edge cases** and a cocotb driver, each phase built on previous phases and thus everything is **regression tested** every single phase
- Five benchmark programs (standard looping, alternating branching, divisibility check (heavy nested looping), periodic branch, and recursive Fibonacci) measured for CPI and branch-prediction accuracy at two optimization levels
- A working **C toolchain**: RISC-V GCC → `crt0.s` startup → linker script → `.o` → `.elf` → `.hex` → simulated on the real CPU, running actual compiled programs (loops, recursion) rather than only hand-written assembly
- Synthesized and physically implemented through the full RTL-to-GDSII flow using [OpenLane2](#) and the [SKY130 PDK](#), ran locally in a Docker + WSL environment

Architecture

Like my single cycle RV32I, `soc_top` wraps the pipelined core (`rv32i_core`) together with separate instruction (ROM) and data memory (RAM) modules, same top-level shape as the single-cycle design.

Microarchitecture diagram (zoom in if needed):

 5-stage pipeline diagram

Pipeline stages:

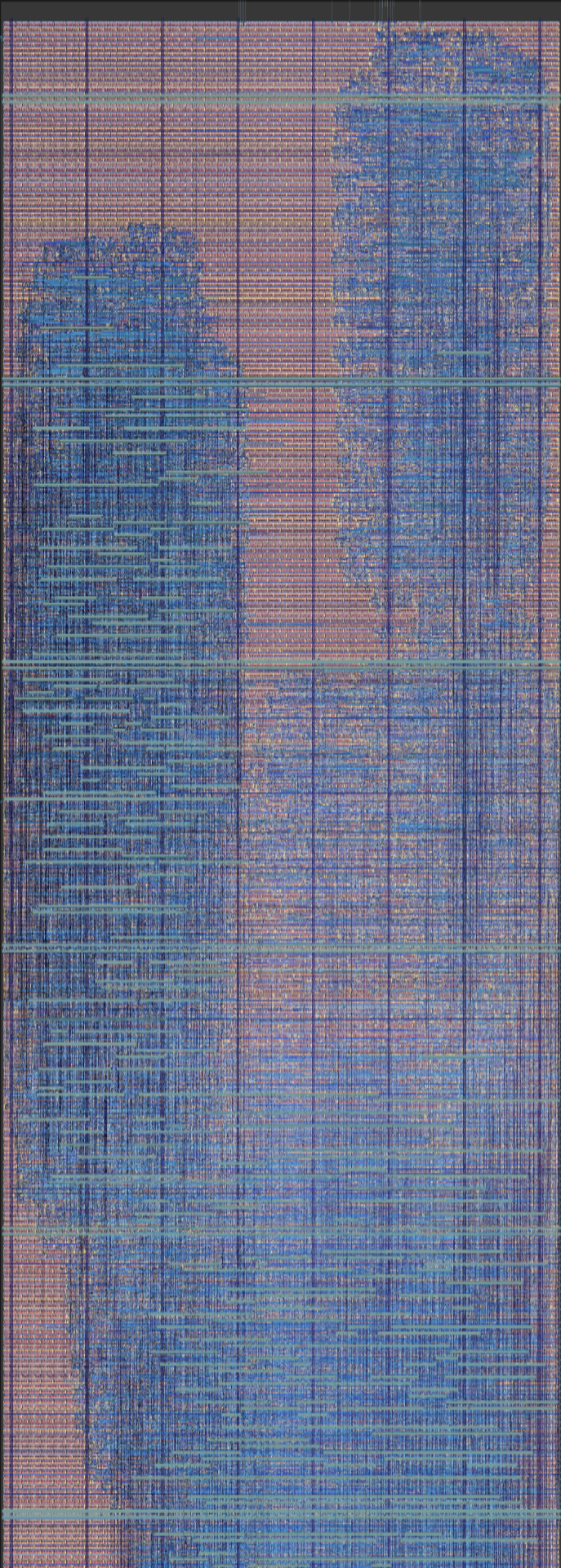
- **IF (Fetch):** Program Counter → Instruction Memory, with the branch predictor's read port consulted the same cycle to speculatively redirect fetch for previously-seen branches/jumps
- **ID (Decode):** Control Unit + Immediate Generator + Register File read, **plus branch/jump resolution** (`branch_comp`, branch-target adder, and a dedicated `jalr` target adder) — moved here from EX to cut the misprediction penalty from 2 cycles to 1
- **EX (Execute):** ALU, fed by a dedicated EX-stage forwarding unit (`forwarding_unit`) resolving RAW hazards from EX/MEM and MEM/WB
- **MEM (Memory Access):** Data Memory, Load Filter, Store Mask, plus a MEM-stage forwarding unit (`mem_forwarding_unit`) specifically for the `lw`-immediately-followed-by-`sw` case
- **WB (Writeback):** A 2-way mux selects between memory-loaded data and an already-resolved "actual result" value (see [Major Debugging Findings](#) for why this collapsed from a 4-way mux)

New pipeline registers: `if_id_reg.v`, `id_ex_reg.v`, `ex_mem_reg.v`, `mem_wb_reg.v`, with appropriate ones supporting the `freeze`/`bubble`/`flush` control inputs intaking the stall and flush logic described below.

GDS Render

rv32l_core (rv32l_core)

- KEYS
- 1: Hide Fill, Decap, Tap cells
 - 2: Hide top cell geometry
 - 3: Isolate selection / back
 - 4: Zoom to selection



▼ Controls

> View Settings

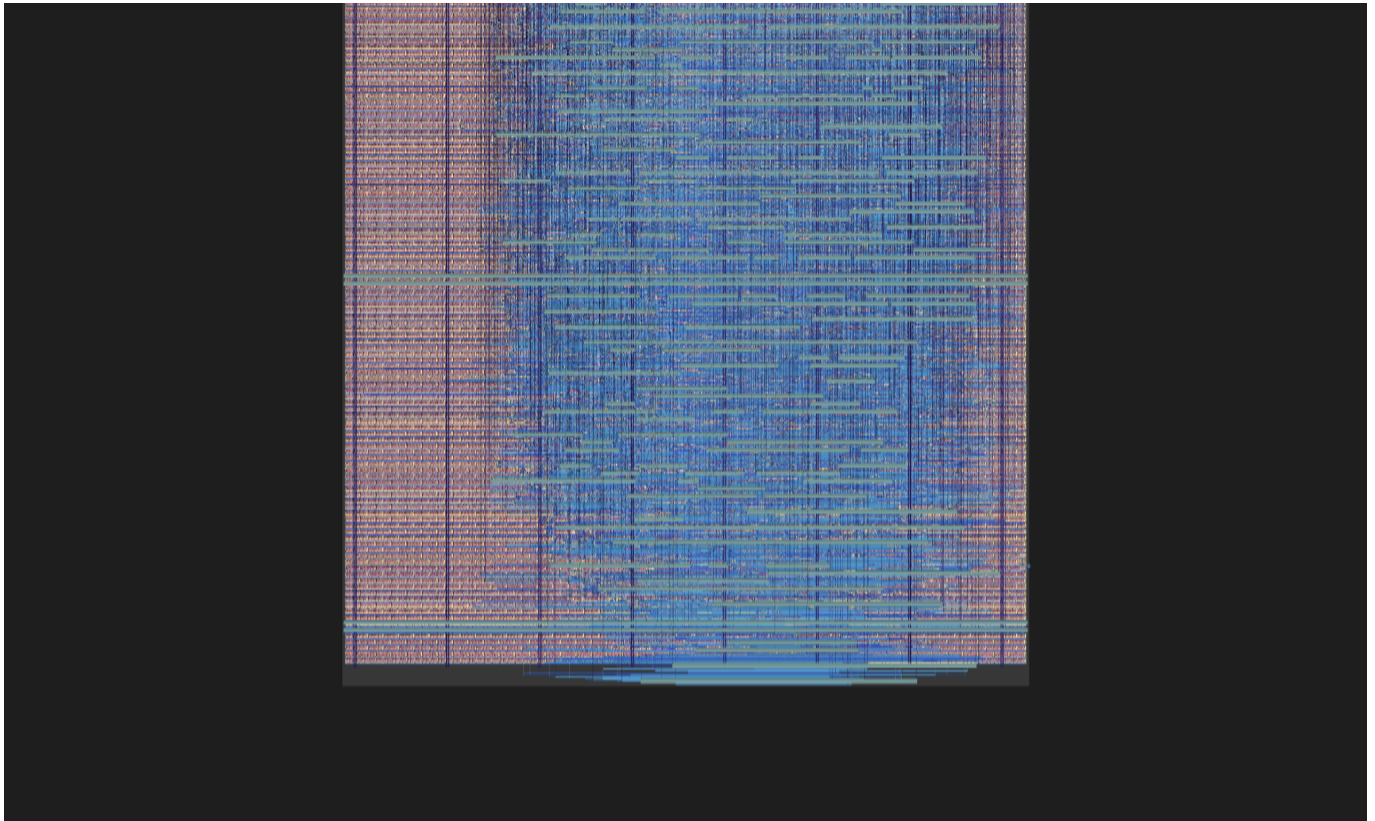
▼ Layers

ALL	✓
substrate	✓
mwel	✓
diff	✓
poly	✓
icon	✓
li1	✓
mcon	✓
met1	✓
via	✓
met2	✓
via2	✓
met3	✓
via3	✓
capm	✓
met4	✓
cap2m	✓
via4	✓
met5	✓

> Cells/Instances

> Performance

> Experimental



*Visibly, the logic cells are not using the full area of the core. I described the reason behind this in later sections: [ASIC Implementation](#), the **Known Limitation** section under that, and [Routing congestion tradeoff](#).

Design

A few notable pipeline-specific design decisions worth calling out (in addition to everything carried over from the single-cycle core):

- **Branch/jump resolution moved to ID**, cutting the control-hazard penalty from 2 cycles (EX-stage resolution) to 1 cycle. This required its own forwarding path (`id_forwarding_unit`) since ID-stage resolution introduced a brand-new hazard case: a producer still live in EX (not yet even reached `ex_mem_reg`) feeding a consumer one stage earlier than any EX-stage consumer ever could. This also requires the hazard unit to account for this change, as stated in the next point.
- **Two-tier load-use stall detection** in `hazard_unit`: the original EX-stage-load-into-ID-stage-consumer stall (sufficient for ALU consumers) plus a second, branch-specific stall condition catching a load that's advanced to MEM while a *branch* still needs it in ID, since branch resolution in ID needs the value one pipeline stage earlier than an ALU consumer would.
- **mem_to_reg-resolved value collapse**: instead of carrying `alu_result`, `pc_plus_4`, and `imm_gen_out` as three separate raw values through `ex_mem_reg/mem_wb_reg` and re-deriving "which one is the real answer" at every consumer (EX forwarding, MEM forwarding, final writeback), the `mem_to_reg`-based resolution now happens once in EX and the *resolved* value is what gets latched forward. This was a real, STA-driven optimization, see [ASIC Implementation](#).
- **Two-stage halt latch**: `halted` (from `halt_d`, fires the instant `ecall/ebreak` is decoded in ID, freezing fetch immediately) and `fully_halted` (from `mem_wb_halt`, fires once the halt instruction actually travels through the entire pipeline and thus allowing any unfinished instructions to properly wrap up,

this is what the top-level `halt` output reflects, giving external logic/testbenches a stable, permanent signal to check).

Module list (`rtl/`)

Top modules: `soc_top.v` · `rv32i_core.v`

Pipeline registers: `if_id_reg.v` · `id_ex_reg.v` · `ex_mem_reg.v` · `mem_wb_reg.v`

Hazard/forwarding: `forwarding_unit.v` · `id_forwarding_unit.v` · `mem_forwarding_unit.v` · `hazard_unit.v`

Branch prediction & control: `branch_predictor.v` · `halt_latch.v`

Carried over, unmodified from R32-SC: `alu.v` · `alu_control.v` · `branch_comp.v` · `control_unit.v` · `data_mem.v` · `imm_gen.v` · `instruction_mem.v` · `load_filter.v` · `pc.v` · `reg_file.v` · `store_mask.v`

Pipeline Hazards & Forwarding

Data hazards (RAW): three separate, purpose-built forwarding units, each covering a different stage/consumer pairing:

Unit	Consumer	Candidates	Purpose
<code>forwarding_unit</code>	ALU operands in EX	EX/MEM, MEM/WB	Original Phase-2 forwarding; covers ordinary ALU-consuming instructions
<code>id_forwarding_unit</code>	<code>branch_comp</code> / branch-target math in ID	live EX, EX/MEM	New candidate introduced by moving branch resolution to ID, the closest possible producer for a 0-NOP gap is now <i>still executing in EX</i> , not yet latched anywhere
<code>mem_forwarding_unit</code>	Store data in MEM	MEM/WB	Specifically resolves <code>lw</code> immediately followed by <code>sw</code> using the same register, saving a stall

The MEM/WB-stage case (producer's writeback and consumer's read landing on the same cycle) is handled for free by `reg_file`'s own same-cycle write-first/read-second internal bypass mux.

Data hazards (load-use): `hazard_unit` detects a load in EX (or, for branch consumers specifically, a load that's in EX or MEM) needing its result before it's ready, and stalls the pipeline by freezing `pc/if_id_reg` while inserting a bubble into `id_ex_reg`. This buying exactly enough cycles for forwarding to pick up the value once it's actually available.

Control hazards: resolved via `flush`, gated on `!stall` (during a stall PC and ID/IF reg should be completely frozen) and on `mispredicted` (see below in [Branch Prediction](#)) rather than firing on every single branch a CPU without dynamic branch predictor would, meaning a correctly-predicted branch causes **zero** flush.

x0 exclusion: every forwarding/hazard comparator explicitly excludes `rd == x0` matches, verified with dedicated test cases confirming no accidental forwarding occurs into or out of the hardwired-zero register.

Branch Prediction

A from-scratch **2-bit saturating-counter predictor backed by a 64-entry, direct-mapped Branch Target Buffer (BTB)**.

Table structure, indexed by `fetch_pc[7:2]` (6-bit index, $2^6 = 64$ entries), tagged by `fetch_pc[31:8]` (24-bit tag, because $32 \text{ (address)} - 6 \text{ (index)} - 2 \text{ (bottom 2 bits omitted since instructions are byte-aligned)} = 24$), to detect and correctly handle index aliasing between unrelated branch addresses:

- `valid`, has this slot ever been written
- `tag`, confirms the entry actually belongs to this address, not an aliased collision
- `target`, the last known branch/jump target for this address
- `counter`, 2-bit saturating bias counter; top bit determines the taken/not-taken prediction

Update policy: on a genuine tag match, the counter increments/decrements toward the observed outcome (saturating at `00/11`); on a fresh occupant (or aliased eviction), the counter resets to a weak bias matching that first real observation, rather than inheriting a previous occupant's unrelated history.

Misprediction detection & flush: `mispredicted` compares the prediction carried through `if_id_reg` (`if_id_predicted_taken/target`) against the freshly-resolved ground truth in ID (`real_taken/real_target`). This covers both direction mispredicts and target mispredicts, and the compound case of directions agreeing but the `target` being wrong. `pc_next` is gated on `mispredicted` (not on `take_branch` directly) specifically to avoid a bug I found while writing RTL: redundantly re-targeting an already-correctly-predicted branch's own address a second time after it resolves (see [Major Debugging Findings](#)).

Measured accuracy: see [Performance Benchmarks](#). This ranges from ~60% on a deliberately worst-case alternating branch up to ~99.97% on predictable loops.

Real Halt Implementation

The single-cycle core's `halt` was purely an informational signal at WB. The core kept fetching and executing forever afterward, and every test program relied on a hand-written `beq x0,x0,halt` self-loop to stay stable.

R32-P5 implements **real halt**: `ecall/ebreak` decoded in ID immediately and permanently freezes fetch (`halted`, sticky, latches on `halt_d`), while a second latch (`fully_halted`, on `mem_wb_halt`) only asserts once the halt instruction actually retires through the full pipeline. This guarantees every instruction scheduled before the halt has genuinely completed before the top-level `halt` output (driven by `fully_halted`) is trusted externally. `crt0.s` and every testbench now use `ecall / ebreak` as the real, hardware-enforced end-of-program convention.

Verification

Module-level (SystemVerilog + Icarus Verilog + cocotb)

Reused directly from the single-cycle design for every unmodified leaf module. New modules (`branch_predictor`, `hazard_unit`, `forwarding_unit`, `id_forwarding_unit`, `mem_forwarding_unit`,

`halt_latch`) were verified via directed, full-branch-coverage assembly test cases integrated into the phase testbenches below, rather than standalone constrained-random testbenches. This is because each unit's functional surface (a handful of priority/comparator cases) is small enough that directed coverage is both sufficient and more informative than random sampling.

Full-core, phase-by-phase (Python cocotb + Icarus + Makefile + self-checking RISC-V Assembly)

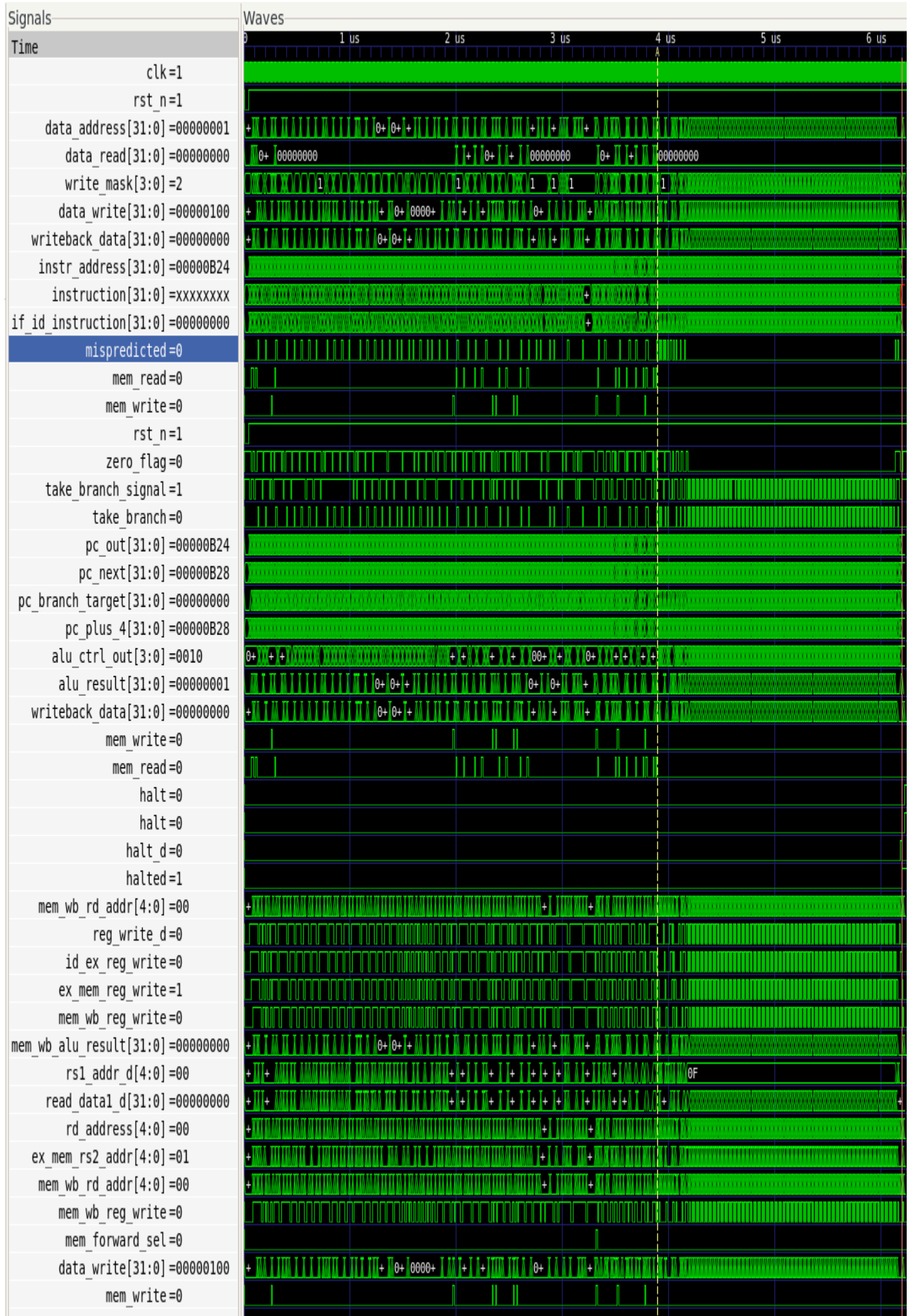
Each pipeline capability was built and verified as its own phase, each with a dedicated self-checking assembly program and cocotb driver. Each phase was built on previous phases and thus everything before is regression tested every single phase.

Error reporting system following the same error code in `x10` convention as the single-cycle core, but now expanded.

Phase	Capability verified
Phase 1	Basic 5-stage datapath cut, no forwarding/hazard/flush (NOP-padded)
Phase 2	EX-stage forwarding, MEM-stage store forwarding, x0 exclusion, priority (EX/MEM over MEM/WB)
Phase 3	Load-use hazard detection and stalling, including back-to-back and branch-consumer cases
Phase 4	Control-hazard flushing, branch resolution moved to ID, zero-NOP taken/not-taken/ <code>jal/jalr</code> correctness
Phase 5	Dynamic branch predictor: warm-up/steady-state accuracy, misprediction correcting ability

Example of a Phase 5 loop-warmup testbench measuring real per-iteration cycle cost (predictor cold-miss vs. steady-state):

Example full core testbench generated waveform:



Full error reporting table showing every case tested:

Epic Error Codes Meaning Table Below

Error code: 2 = failed to not take branch during invalid beq
Error code: 3 = failed to take branch during valid beq
Error code: 4 = failed addi instruction
Error code: 5 = failed to lw and sw properly
Error code: 6 = failed andi instruction
Error code: 7 = failed ori instruction
Error code: 8 = failed xor instruction
Error code: 9 = failed xori instruction
Error code: 10 = failed sll instruction (logical left)
Error code: 11 = failed slli instruction (logical left immediate)
Error code: 12 = failed srl instruction (logical right)
Error code: 13 = failed srli instruction (logical right immediate)
Error code: 14 = failed sra instruction (arithmetic right sign not preserved)
Error code: 15 = failed srar instruction (arithmetic right immediate sign not preserved)
Error code: 16 = failed slt instruction (signed comparison)
Error code: 17 = failed sltu instruction (unsigned comparison)
Error code: 18 = failed slti instruction (signed immediate comparison)
Error code: 19 = failed sltiu instruction (unsigned immediate comparison)
Error code: 20 = failed to take branch during valid bne
Error code: 21 = failed to not take branch during invalid bne
Error code: 22 = failed to take branch during valid blt (signed)
Error code: 23 = failed to not take branch during invalid bltu (unsigned)

```
|Error code: 24 = failed to not take branch during invalid bge (signed) |
```

```
-----  
|Error code: 25 = failed to take branch during valid bgeu (unsigned) |
```

Epic Error Codes Meaning Table Above

Compiled C toolchain

Toolchain: RISC-V GCC (`-march=rv32i -mabi=ilp32 -nostdlib -nostartfiles`) compiles C source, a hand-written `crt0.s` initializes the stack pointer and calls `main()`, a custom linker script (`link.ld`) places `.text/.data` into the CPU's two separate physical memory spaces, and `objcopy -O verilog` produces the final `$readmemh`-format hex — the same format hand-written assembly tests have used since Phase 1.

First working program: a trivial `int main() { return 1 + 2; }`, verified using cocotb (`x10 == 3`) before moving to the loop-heavy benchmarks below.

Known limitations

- Full-core differential testing against a reference ISA simulator (Spike or a self-written Python interpreter) is planned but not yet implemented.
- The official `riscv-arch-test` compliance suite was scoped but not integrated. RISCOF requires a signature-dump testbench mechanism and target-specific `RVMODEL_*` macros that weren't built out in this pass; individual official test files are a planned lighter-weight alternative.
- A dedicated BTB-aliasing stress test (two colliding addresses, confirming tag-mismatch correctly falls back to a cold miss) was reasoned through and proven correct by construction (index+tag together reconstruct the full address) but not exercised with a purpose-built assembly test.

Performance Benchmarks

Five C programs, compiled with GCC with both flag `-O0` (unoptimized) and `-O2` (production-representative), each measuring CPI (cycles / approximate retired instructions, using stall/flush cycle counts as the correction term) and branch-prediction accuracy directly from live RTL signals (`mispredicted`, `stall`, `flush`) during simulation:

Test	Description	-O0 CPI	-O0 Accuracy	-O2 CPI	-O2 Accuracy
<code>loop1</code>	Simple counting loop (best case, highly predictable)	1.200	99.95%	1.000	99.96%
<code>loop2</code>	Alternating <code>if/else</code> every iteration (near-worst-case for a 2-bit counter)	1.296	59.99%	1.000	99.97%
<code>loop3</code>	Divisibility check via repeated-subtraction <code>mod()</code> , inlined 3x per iteration, called functions's interior iterations increases as dividend increases	1.426	66.22%	1.012	98.28%
<code>loop4</code>	Period-4 branch pattern (<code>((i&3)!=3)</code>)	1.226	87.50%	1.160	87.50%
<code>fib(15)</code>	Recursive Fibonacci, stresses nested call/return stack usage	1.227	67.11%	1.032	76.54%
Average		1.275	76.15%	1.041	92.45%

Notable findings from this benchmarking pass:

- `-00` numbers are closer to a *hazard-logic stress test* than realistic performance, GCC keeps every loop variable in RAM, maximizing load-use stalls due to unoptimized assembly code; `-02` numbers are closer to representative real-world performance.
- `loop2`'s accuracy jump (60% → 99.97%) under `-02` is because of GCC's loop unrolling restructured the alternating branch into a different, much more predictable control-flow pattern for the CPU entirely after inspecting the compiled assembly code. This is something completely new to me and I was genuinely impressed by this level of compiler optimization when I found out.
- A naive `int main(){ for(...) count++; return count; }` loop is fully eliminated by `-02` (calculating final value directly during compile time instead of running the loop) unless the loop variable is marked `volatile`. The first version of `loop1` completed in 14 cycles for exactly this reason before the fix.

Overall, these testing showed me my CPU's level of performance, but more importantly allowed me to learn more about compiler optimization, compiler toolchain, and C language itself.

Major Debugging Findings

A few of the more substantial bugs I found and fixed during this project, worth documenting for hitting similar issues in the future:

- **Forwarding Unit assumed `alu_result` was always the final answer that should be forwarded.** For `jal/jalr/lui`, the real writeback value is `pc_plus_4` or the raw immediate, not the ALU's output (which is meaningless for these instruction types, since their encodings reuse the `rs1/rs2` bit positions for other purposes). A `jal` immediately followed by `jalr` using its link register forwarded this garbage value, sending the CPU's PC to `0x0`. Fixed by building `ex_actual_result/ex_mem_actual_result` — `mem_to_reg`-aware resolution wires and forwarding *those* instead of raw `alu_result` everywhere.
- **A redundant PC re-target on correctly-predicted branches.** Before `pc_next` was gated on `mispredicted`, a correctly-predicted taken branch's own resolution in ID would still unconditionally re-select its branch target a second time, causing the very next instruction fetched by the predictor's correct speculation to be *re-fetched* a second time, causing potential data corruption and is not just a performance loss, caught by hand-tracing cycle-by-cycle PC values on paper before it ever showed up as a wrong testbench result.
- **Single-bit-wide `reg` array `[0:63]` patterns are unreliable to synthesize.** `branch_predictor`'s `valid` array (1 bit per entry) crashed Yosys's partway through synthesis, while the wider `tag/target/counter` arrays synthesized cleanly. Yosys's per-element-unrolling fallback path for narrow arrays was unreliable. Fixed by flattening every array into a single wide vector (`reg [63:0] valid, reg [24*64-1:0] tag_flat`, indexed via `+`: part-selects) rather than using genuine multi-element arrays at all.
- **OR-ing a synchronous signal into the same condition as an async reset breaks synthesis.** `if (!rst_n || flush)` mixes `rst_n` (async, in the sensitivity list) with `flush/bubble` (ordinary synchronous signals) in one combined condition confused Yosys (error message: "`Multiple edge sensitive events found for this signal!`"), even though Icarus simulated it correctly. Fixed by

separating every pipeline register's reset logic into distinct `if (!rst_n) ... else if (bubble) ... else if (flush) ...` branches.

- **A one-cycle race between `halt_d` and the sticky `halted` latch.** Since `halted` only updates on a clock edge, the exact cycle `ecall` is first decoded still has `halted == 0`, briefly allowing one more wrong-path fetch through before the freeze engages. I resolved this by including `halt_d` itself (the immediate, same-cycle signal) directly in `if_id_reg`'s/`pc`'s freeze and flush conditions, not just the one-cycle-delayed `halted`.

ASIC Implementation

Synthesized end-to-end (RTL → GDSII) using **OpenLane2** against the **SKY130** open-source PDK, run locally via WSL + Docker, same approach as the single-cycle core.

Synthesis strategy selection

All nine available `SYNTH_STRATEGY` options were compared directly using OpenLane2's built-in `SynthesisExploration` flow at an initial 28 ns clock target, rather than assuming any one strategy would be best:

<code>SYNTH_STRATEGY</code>	Gates	Area (μm^2)	Worst Setup Slack (ns)	Total -ve Setup Slack (ns)
AREA 0	19,703	284,406.5	-14.34	-3,608.30
AREA 1	19,866	284,178.8	-9.36	-2,321.80
AREA 2	19,465	281,498.7	-12.53	-3,831.75
AREA 3	29,608	330,316.8	+7.24	0.0
DELAY 0	20,694	301,674.3	-9.17	-3,360.45
DELAY 1	20,348	297,328.9	-11.64	-914.95
DELAY 2	20,329	297,554.1	-10.72	-717.92
DELAY 3	20,502	299,627.4	-14.63	-1,240.08
DELAY 4	22,932	307,743.9	-15.23	-1,363.58

AREA 3 was the only strategy meeting timing at all. Despite the name, it apparently uses aggressive algebraic logic-collapsing that empirically outperformed every **DELAY** strategy for control-logic-heavy design (like this pipelined CPU!).

I came across this whilst researching OpenLane2 config optimization results, see the [research paper](#) I found.

The tradeoff, very ironically (**AREA 3** means most aggressively minimizing logic area in OpenLane2), is a substantially higher gate count, which directly caused the routing congestion issues described in **Routing congestion tradeoff** 2 sections below.

Results

Metric	Value
Clock period	20 ns
Clock speed	50 MHz (<i>~40% faster than the single-cycle core's 35.7 MHz</i>)
Total cell count	27,953
Flip-flops	5,261
Wire count	27,861
Total wire length	3,032,556 μm (~ 3.03 m)
Core Area	1,287,160 μm^2
Logic Area	376,233 μm^2
Core Utilization	29.2%
DRC	0 violations
LVS	Circuits match
Setup & hold timing	Met at all corners @ 20 ns

Routing congestion tradeoff

AREA 3's much higher gate count ($\sim 30\text{k}$ vs. $\sim 19\text{-}23\text{k}$ for every other strategy) repeatedly triggered **GRT-0118** global routing congestion failures at the density settings that worked fine for other strategies. Resolving this took hours of ASIC flow runs with different combinations of config parameters and physical tradeoffs rather than a single fix:

- **PL_TARGET_DENSITY** loosened to **0.35** and **FP_CORE_UTIL** to **25**, deliberately sacrificing die efficiency (down to $\sim 29\%$ utilization) to give the router physical room
- **SYNTH_MAX_FANOUT/MAX_FANOUT_CONSTRAINT** widened to **16** (up from a much tighter, over-aggressive **4** tried earlier) to avoid an explosive buffer-tree cell-count increase
- **PL_ROUTABILITY_DRIVEN** enabled, OpenROAD's placer-level congestion-aware cell inflation, targeting local hotspots directly rather than uniform density changes
- **GRT_ADJUSTMENT** set to **0.3**, reserving explicit extra margin on routing tracks at the global-routing stage itself

Critical path optimization

Post-route STA identified the critical path originating from **ex_mem_reg**'s **mem_to_reg_out**, propagating through a 4-way value-resolution mux that was being redundantly re-computed at three separate pipeline stages (see [Major Debugging Findings](#)). Resolving the value once in EX and carrying the resolved result forward rather than re-deriving it at MEM and again at WB collapsed two of the three redundant 4-way muxes down to 2-way, directly shortening this path and making it no longer the critical path.

A second, distinct critical path was subsequently identified through **mem_wb_rd_addr**'s address-comparison fanout (feeding both **forwarding_unit**'s and **reg_file**'s independent equality checks). This could be a candidate for future optimization rather than resolved in this pass, since it's structurally necessary comparison logic rather than a redundant computation.

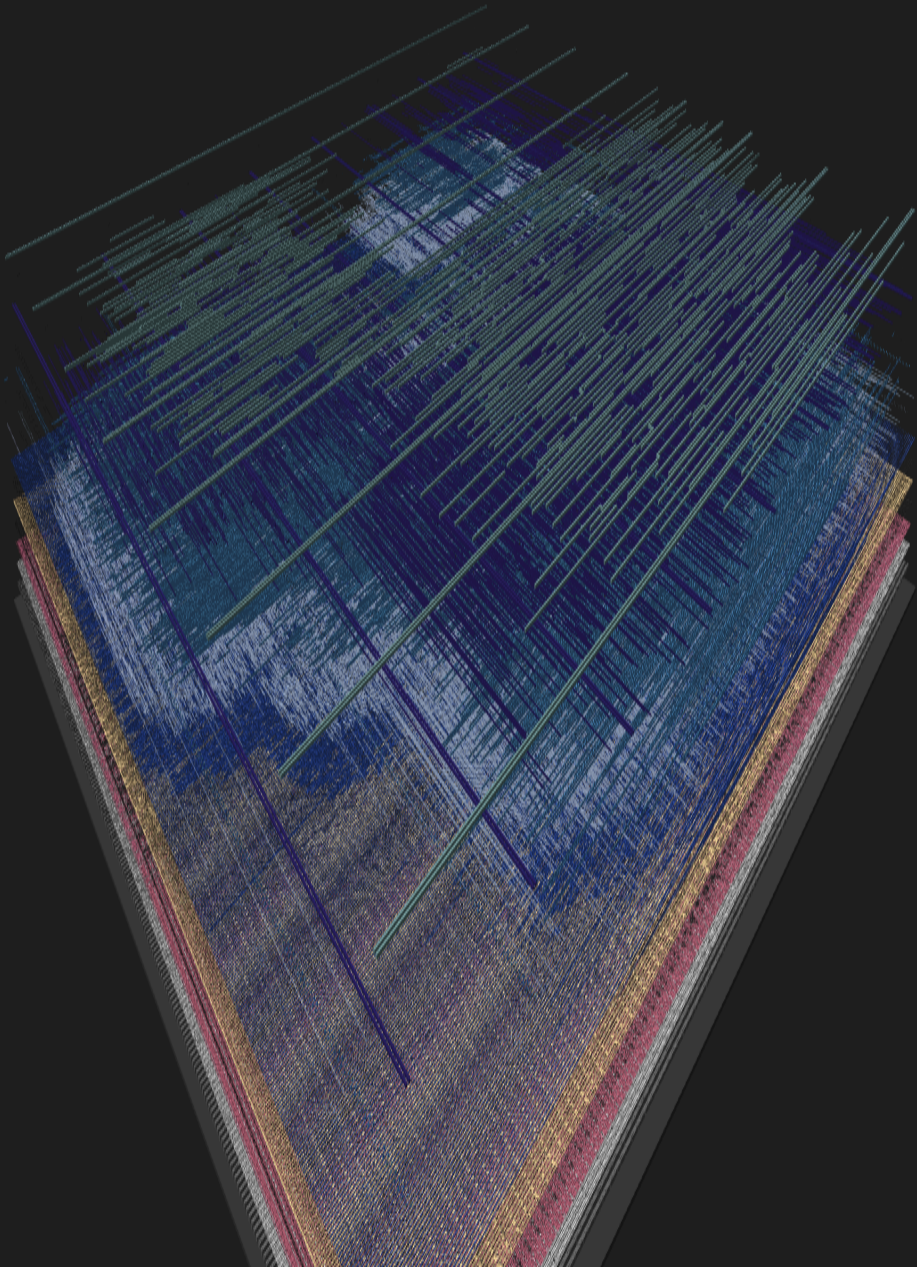
Final OpenLane2 config.json setup

```
{
  "DESIGN_NAME": "rv32i_core",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 20,
  "PNR_SDC_FILE": "dir::src/constraint.sdc",
  "SIGNOFF_SDC_FILE": "dir::src/constraint.sdc",
  "SYNTH_STRATEGY": "AREA 3",
  "SYNTH_SIZING": 1,
  "DIODE_INSERTION_STRATEGY": 3,
  "NUM_THREADS": 12,
  "ROUTING_CORES": 12,
  "KLAYOUT_XOR_THREADS": 12,
  "SYNTH_MAX_FANOUT": 16,
  "MAX_FANOUT_CONSTRAINT": 16,
  "SYNTH_ABC_DFF": 1,
  "PL_TARGET_DENSITY": 0.35,
  "FP_CORE_UTIL": 25,
  "GRT_ADJUSTMENT": 0.3,
  "FP_ASPECT_RATIO": 1.0,
  "PL_ROUTABILITY_DRIVEN": 1,
  "PL_RESIZER_TIMING_OPTIMIZATIONS": 1,
  "SYNTH_SHARE_RESOURCES": 0
}
```

Disassembled GDS render:

rv32l_core (rv32l_core)

- KEYS
- 1: Hide Fill, Decap, Top cells
 - 2: Hide top cell geometry
 - 3: Isolate selection / back
 - 4: Zoom to selection



▼ Controls

▼ View Settings

Isolate selection / Back

Zoom selection

Filler cells ☒

Top cell geometry ☒

▼ Layers

ALL ☒

substrate ☒

mwel ☒

diff ☒

poly ☒

icon ☒

li1 ☒

mcon ☒

met1 ☒

via ☒

met2 ☒

via2 ☒

met3 ☒

via3 ☒

capm ☒

met4 ☒

cap2m ☒

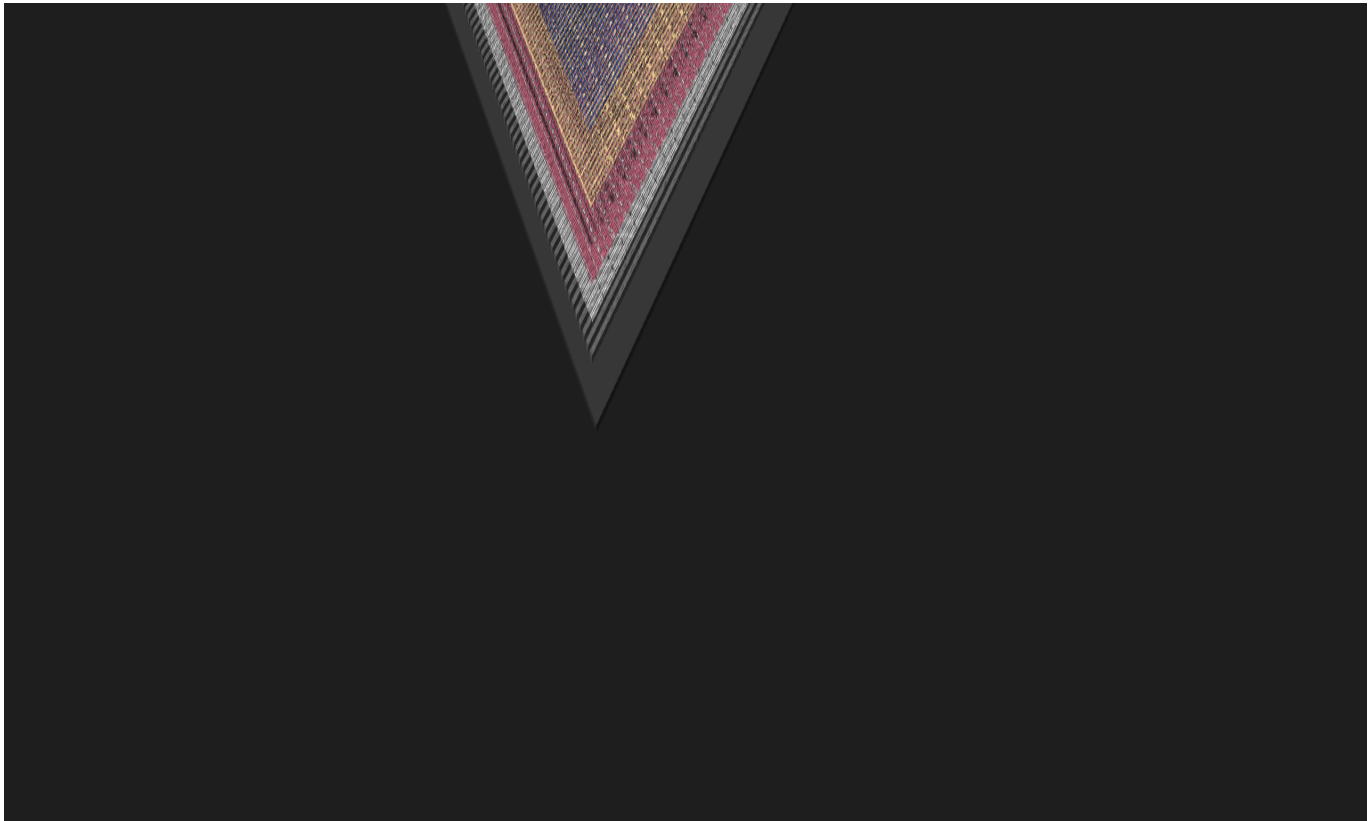
via4 ☒

met5 ☒

> Cells/Instances

> Performance

> Experimental



Cell breakdown:

Category	Cell Types	Count
Combinational (AOI/OAI compound gates)	a2111o, a2111oi, a211o, a211oi, a21bo, a21boi, a21o, a21oi, a221o, a221oi, a22o, a22oi, a2bb2o, a2bb2oi, a311o, a31o, a31oi, a32o, a41o, o2111a, o2111ai, o211a, o211ai, o21a, o21ai, o21ba, o21bai, o221a, o221ai, o22a, o22ai, o2bb2a, o2bb2ai, o311a, o311ai, o31a, o31ai, o32a, o41a	9,251
Flip-Flops	dfrtp	5,261
NAND	nand2, nand2b, nand3, nand3b	3,483
Buffer	buf, bufbuf, bufinv	2,473
OR	or2, or2b, or3, or3b, or4, or4b	2,200
NOR	nor2, nor3, nor3b	1,384
Inverter	inv	1,374
Multiplexer	mux2	1,357
AND	and2, and2b, and3, and3b, and4, and4b	1,104
XOR/XNOR	xnor2, xor2	66
Total		27,953

Known limitations

- Max slew / max cap violations remain in the **ss** (slow-slow) process corner, similar in category to the single-cycle core's residual signal-integrity findings. Thgis does not block DRC/LVS/timing signoff, but a real consideration for an actual fabricated chip.
- Core utilization (~30%) is lower than the single-cycle design's 47.3%, which is the cost of choosing **AREA 3** for its timing win. Revisiting this would mean either accepting a larger die or finding a synthesis-strategy/RTL combination that gets **AREA 3**-level timing without **AREA 3**-level gate count. Or alternatively, more ASIC flow iterations for more fine tuned and optimized OpenLane2 configuration flags.

Repository Structure

```

|
├── rtl/
│   ├── rv32i_core.v          # Top level pipelined CPU module
│   ├── if_id_reg.v          # IF/ID pipeline register (freeze/flush)
│   ├── id_ex_reg.v          # ID/EX pipeline register (bubble)
│   ├── ex_mem_reg.v         # EX/MEM pipeline register
│   ├── mem_wb_reg.v         # MEM/WB pipeline register
│   ├── forwarding_unit.v     # EX-stage forwarding (EX/MEM, MEM/WB)
│   └── id_forwarding_unit.v  # ID-stage (branch_comp) forwarding (live EX,
EX/MEM)
│   ├── mem_forwarding_unit.v # MEM-stage store-data forwarding (specifically
for lw followed by sw)
│   ├── hazard_unit.v        # Load-use stall detection (two-tier)
│   ├── branch_predictor.v   # 2-bit saturating counter + 64-entry BTB
│   └── halt_latch.v         # Sticky halt latch (used twice: halted,
fully_halted)
│   ├── data_mem.v           # RAM module
│   ├── instruction_mem.v    # ROM module
│   ├── soc_top.v            # SoC routing CPU with RAM and ROM
│   └── all other .v files   # Unmodified modules from R32-SC
├── sim/
│   ├── phase_1/ .. phase_5/ # Each pipeline capability's Makefile + build dir
│   └── c_hello_world/       # First compiled-C bring-up: crt0.s, link.ld,
Makefile
│   ├── loop1/ .. loop4/, fib/ # Compiled-C benchmark folders (same
crt0.s/link.ld pattern)
│   └── cocotb_sim_*/         # Per-phase cocotb/Icarus build + waveform dump
locations
├── tb/
│   ├── modules/             # Per-module SystemVerilog testbenches (unmodified
leaf modules)
│   ├── programs/            # Assembled/compiled program artifacts
(.s/.c/.o/.elf/.hex)
│   └── top/                  # phase_1.py .. phase_5.py, phase_1.s ..
phase_5.s, loop*.py, fib.py cocotb drivers

```

Instruction Set Coverage

Like my single-cycle, full RV32I base instruction set. 40/40 instructions implemented but now correctly pipelined including forwarding/hazard handling for every instruction type.

Category	Instructions
Register-Immediate ALU	<code>ADDI SLTI SLTIU ANDI ORI XORI SLLI SRLI SRAI</code>
Register-Register ALU	<code>ADD SUB SLL SLT SLTU SRL SRA XOR OR AND</code>
Upper Immediate	<code>LUI AUIPC</code>
Loads	<code>LB LH LW LBU LHU</code>
Stores	<code>SB SH SW</code>
Jumps	<code>JAL JALR</code>
Branches	<code>BEQ BNE BLT BGE BLTU BGEU</code>
System	<code>FENCE ECALL EBREAK</code>

Unlike the single-cycle core, `ECALL/EBREAK` now trigger a **real, permanent halt** that stops instruction fetch entirely (see [Real Halt Implementation](#)), rather than only raising an informational signal.

`FENCE` remains implemented as a `NOP`.

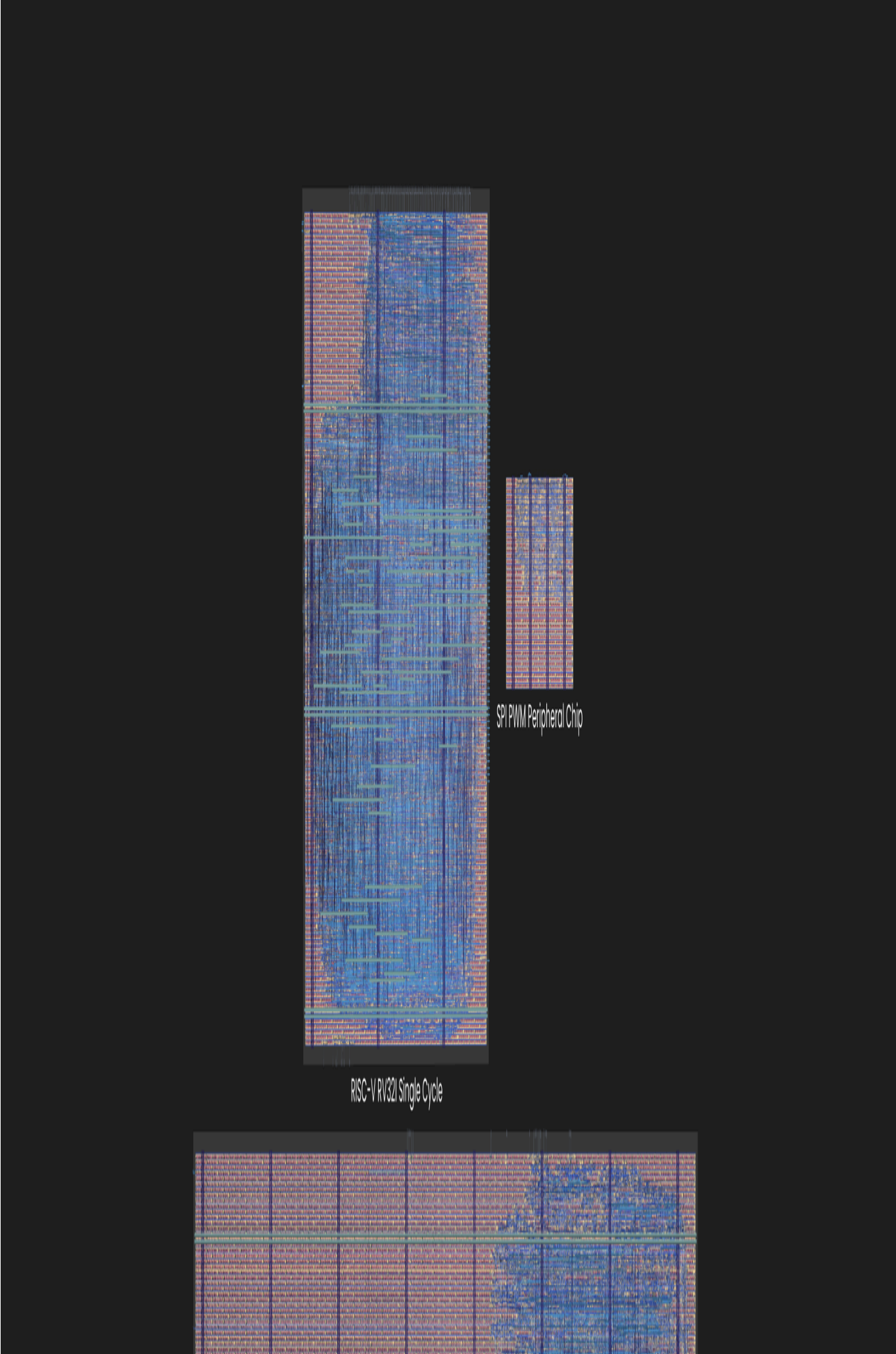
Future Plans

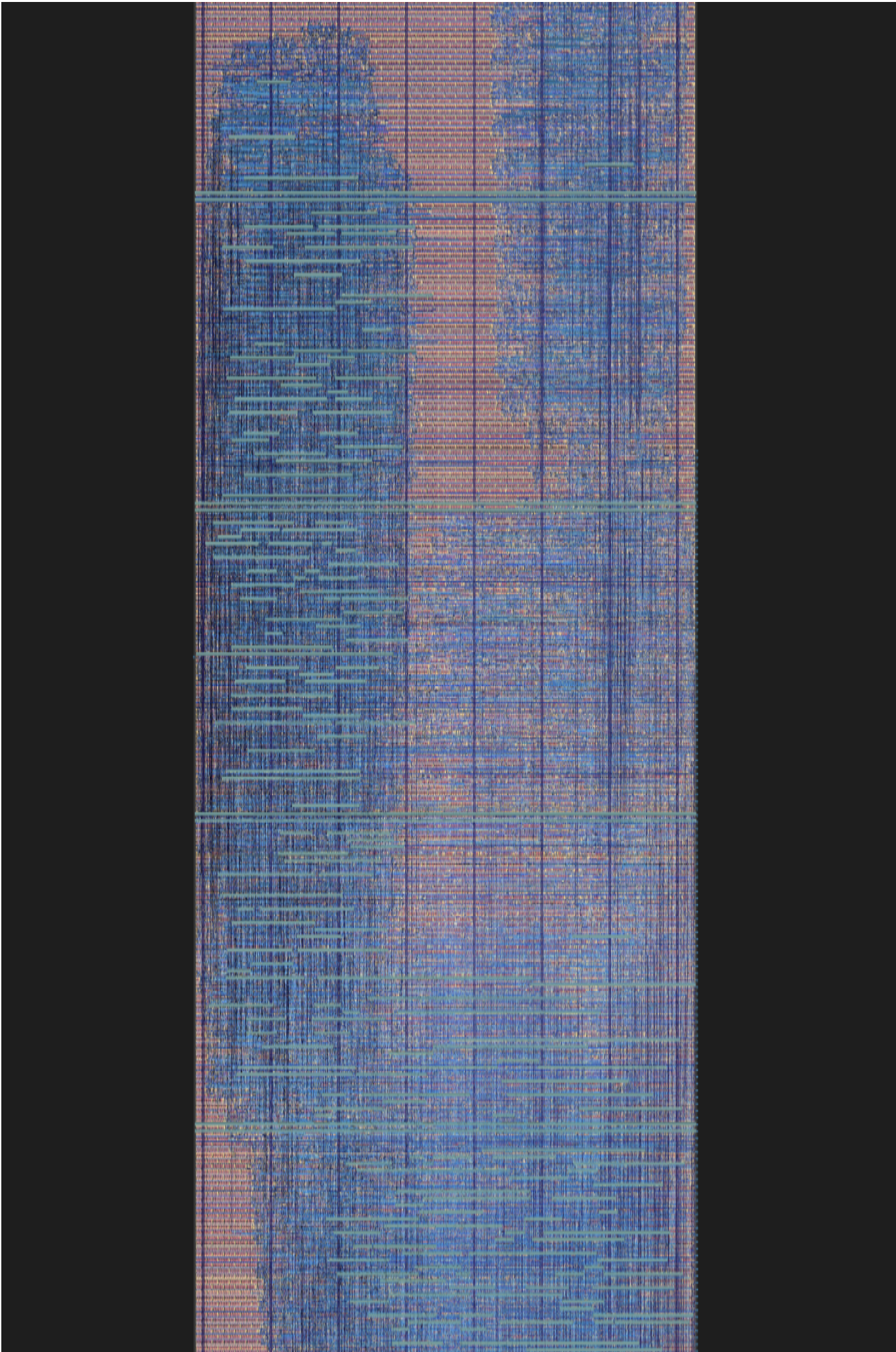
- ☐ Official `riscv-arch-test` compliance suite via RISCOF, including a signature-dump testbench mechanism and target-specific `RVMODEL_*` macros
- ☐ Full-core differential testing against a reference ISA simulator (Spike)
- ☐ Resolve the `mem_wb_rd_addr` forwarding-comparator critical path identified in this pass
- ☐ Optimize RTL to hopefully achieve 100 Mhz+ clock speed
- ☐ Find a synthesis-strategy/RTL combination achieving `AREA 3`-level timing without its full gate-count/utilization cost
- ☐ Dedicated BTB-aliasing stress test and local/global (tournament) branch prediction as a stretch goal
- ☐ FPGA implementation on an AMD Xilinx Artix-7, eventually building toward a VGA-driven SoC
- ☐ Superscalar or out-of-order successor core

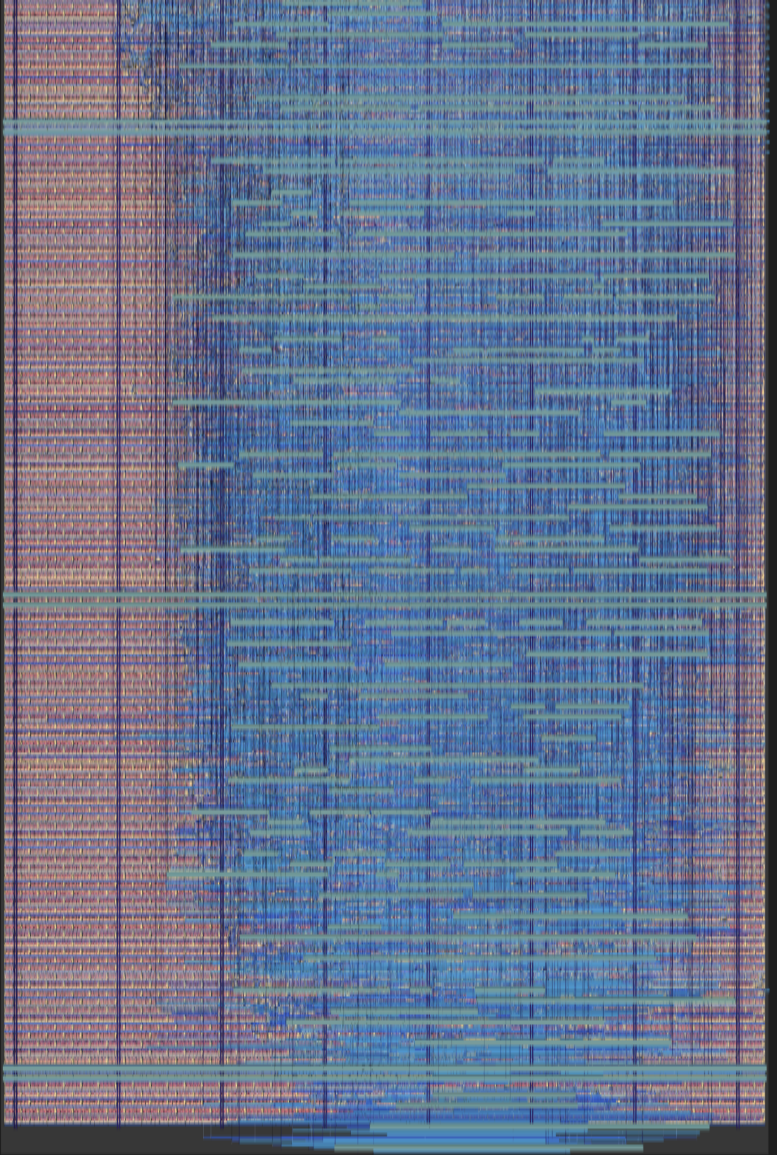
Lastly, a to-scale comparison across all three of my tapeout projects built so far, cus it's fun



(scale estimated through comparing standard filler cell sizes & met4 wire widths in Photopia)







RISC-V RV32I 5-Stage Pipelined

License

Apache-2.0, see [LICENSE](#)

Copyright (c) 2026 Zhiyuan (Jerry) Jiang: frontend & backend VLSI, verification and documentation

All credit for architectural and ISA specification goes to RISC-V International

Resources Used

Computer Organization and Design RISC-V Edition: The Hardware Software Interface - by David A Patterson and John L. Hennessy

Digital Design and Computer Architecture, RISC-V Edition - by David Harris and Sarah Harris

[Automated Parameter Tuning for Timing Closure in OpenROAD/OpenLane: A Comprehensive Multi-Design Grid Search Framework on the SkyWater 130 nm PDK](#) - by Atair Rahman Alvi

[RISC-V Instruction Encoder / Decoder](#) - by LupLab @ University of California, Davis

[WebRISC-V](#) - by Gianfranco Mariotti

[Interactive RISC-V Simulator](#) - by Haoziwan

[1.1 RV32I Base Integer Instruction Set, Version 2.1](#) - by RISC-V International